ジオメトリック・インテリジェンス ケーススタディ補助文書 03

Python のコード —— 曲率の裁定 —— 平坦な確率空間が崩壊するとき

Étale Cohomology

2026 年 11 月　License: CC BY 4.0

本書は、ケーススタディ・ノベル「曲率の裁定 —— 平坦な確率空間が崩壊するとき」（以下、本編）の図を描く Python のコード 7 本について、コードの全文、実行の結果、コードの中で行っている検算、実行の注意をまとめた文書である。

目次

- 第 1 章　はじめに
  - 1.1 コードの置き場所
  - 1.2 実行の方法
  - 1.3 実行の注意
  - 1.4 一覧
- 第 2 章　PY-01：通貨のグラフを一周したときの、両替のずれ（離散的なホロノミー）
  - 2.1 本編との関係
  - 2.2 実行の結果
  - 2.3 検算と読み方
  - 2.4 コードの全文
- 第 3 章　PY-02：平穏な市場と危機の市場の、ユークリッド距離とフィッシャー・ラオ距離
  - 3.1 本編との関係
  - 3.2 実行の結果
  - 3.3 検算と読み方
  - 3.4 コードの全文
- 第 4 章　PY-03：何の距離で見るかで、空間の形が変わる
  - 4.1 本編との関係
  - 4.2 実行の結果
  - 4.3 検算と読み方
  - 4.4 コードの全文
- 第 5 章　PY-04：損失の裾の形が変わる —— VaR と期待ショートフォール
  - 5.1 本編との関係
  - 5.2 実行の結果

    - 5.3 検算と読み方
    - 5.4 コードの全文
- 第 6 章　PY-05：k 形式が測るもの、同じ矢印を 2 本並べると面積は 0
    - 6.1 本編との関係
    - 6.2 実行の結果
    - 6.3 検算と読み方
    - 6.4 コードの全文
- 第 7 章　PY-06：相関が 1 に近づくと、散らばりの「体積」がつぶれる
    - 7.1 本編との関係
    - 7.2 実行の結果
    - 7.3 検算と読み方
    - 7.4 コードの全文
- 第 8 章　PY-07：大きな輪のずれを、三角形ごとに分ける（離散版のストークスの定理）
    - 8.1 本編との関係
    - 8.2 実行の結果
    - 8.3 検算と読み方
    - 8.4 コードの全文
- 第 9 章　実行の環境

第 1 章　はじめに

1.1 コードの置き場所

コードは、GitHub の本作のフォルダの code/figures/ にある。

https://github.com/EtaleCohomology/geometric-intelligence-saga/tree/main/case-studies/corporate/hedge-fund-industry/code

1.2 実行の方法

```
pip install -r code/requirements.txt
python code/figures/mucam_py01_currency_loop.py
```

必要なライブラリは、numpy、scipy、matplotlib である（scipy は PY-04 で使う）。図は、コードを実行したフォルダに、JPEG のファイルとして保存される。

1.3 実行の注意

- 日本語のフォント：図の日本語は、Noto Sans CJK JP のフォントで描く。コードは、実行するコンピューターに Noto Sans CJK のフォントがあれば、自動で見つけて使う。ない場合は、フォントを入れるか、コードの中のフォントの名前を書き換える。
- 数値：計算に使う数値は、すべて物語の中の架空の数値である。
- 乱数：乱数を使うコード（PY-04、PY-06）は、乱数の種を固定しているので、実行するたびに同じ図と同じ数値になる。

- 検算：PY-03 と PY-07 は、数学として成り立つべき関係を、コードの中の assert で確かめている。確かめに失敗すると、実行が止まる。

1.4 一覧

図	コード	本編の場面	出力する画像
PY-01	mucam_py01_currency_loop.py	本編の第 2 章：通貨のグラフを一周したときの、両替のずれ（離散的なホロノミー）	mucam_py01a_currency_graph.jpg、mucam_py01b_loop_sum.jpg
PY-02	mucam_py02_fisher_vs_euclid.py	本編の第 4 章：平穏な市場と危機の市場の、ユークリッド距離とフィッシャー・ラオ距離	mucam_py02_fisher_vs_euclid.jpg
PY-03	mucam_py03_wasserstein_flat.py	本編の第 4 章：何の距離で見るかで、空間の形が変わる	mucam_py03_wasserstein_flat.jpg
PY-04	mucam_py04_tail_and_es.py	本編の第 7 章：損失の裾の形が変わる ―― VaR と期待ショートフォール	mucam_py04_tail_and_es.jpg
PY-05	mucam_py05_k_forms.py	本編の第 3 章：k 形式が測るもの、同じ矢印を 2 本並べると面積は 0	mucam_py05a_k_forms.jpg、mucam_py05b_collapse.jpg
PY-06	mucam_py06_volume_collapse.py	本編の第 4 章：相関が 1 に近づくと、散らばりの「体積」がつぶれる	mucam_py06_volume_collapse.jpg
PY-07	mucam_py07_big_loop_triangles.py	本編の第 8 章：大きな輪のずれを、三角形ごとに分ける（離散版のストークスの定理）	mucam_py07a_big_loop_graph.jpg、mucam_py07b_loop_breakdown.jpg

第 2 章　PY-01：通貨のグラフを一周したときの、両替のずれ（離散的なホロノミー）

2.1 本編との関係

本編の第 2 章の場面の図を描く。出力する画像：
mucam_py01a_currency_graph.jpg、mucam_py01b_loop_sum.jpg

2.2 実行の結果

コードを実行すると、次の値を表示する。

```
一周の和 = -0.0074、増減 = -0.74%
```

2.3 検算と読み方

対数レートの和（−0.0044）と、手数料の分（−0.0030）の合計が −0.0074 になる。1 ドルから出発して一周すると、0.9926 ドルで戻る。数値は架空。

2.4 コードの全文

```
"""
曲率の裁定（MUCAM）図PY-01a・PY-01b：通貨のグラフを一周したときの、両替のずれ（離散的なホロノミー）
PY-01a：通貨を点、両替の経路を線とするグラフ（一周の経路と、各辺の対数レートと手数料）
PY-01b：ドル→人民元→タイバーツ→ドルの一周の計算の表
数値は、物語の中の架空の数値である。
"""
import math
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib import font_manager
from matplotlib.patches import FancyArrowPatch

for f in font_manager.findSystemFonts():
    if "NotoSansCJK" in f and ("Regular" in f or "Bold" in f):
        font_manager.fontManager.addfont(f)
plt.rcParams["font.family"] = "Noto Sans CJK JP"

# ---- 物語の中の架空の数値 ----
loop = [
    ("USD", "CNY", 7.1000, 0.0010),
    ("CNY", "THB", 4.9500, 0.0005),
    ("THB", "USD", 1 / 35.30, 0.0015),
]
rows, total = [], 0.0
for a, b, rate, fee in loop:
    lr, lf = math.log(rate), math.log(1 - fee)
```

```
    rows.append((a, b, rate, lr, lf)); total += lr + lf
gain = math.exp(total) - 1

NAVY, GRAY, TEAL, GOLD, RED = "#1e2a3a", "#9aa3ad", "#1f8a8a",
"#c8962e", "#a0471e"

# ================= PY-01a：通貨のグラフ=================
pos = {"USD": (0.0, 1.05), "CNY": (1.35, -0.45), "THB": (-1.35, -
0.45), "AED": (0.0, -1.7)}
name = {"USD": "米ドル\nUSD", "CNY": "人民元\nCNY", "THB": "タイバーツ\
nTHB", "AED": "UAE\nディルハム\nAED"}
fig = plt.figure(figsize=(16, 9), dpi=120)
ax = fig.add_axes([0.0, 0.10, 1.0, 0.80]); ax.set_aspect("equal");
ax.axis("off")
ax.set_xlim(-3.2, 3.2); ax.set_ylim(-2.2, 1.6)

def edge(a, b, color, lw, ls="-", rad=0.0, arrow=True, z=1):
    ax.add_patch(FancyArrowPatch(pos[a], pos[b],
connectionstyle=f"arc3,rad={rad}",
                 arrowstyle="-|>" if arrow else "-",
mutation_scale=34,
                 color=color, lw=lw, ls=ls, shrinkA=58, shrinkB=58,
zorder=z))

edge("USD", "AED", GRAY, 3, arrow=False)
edge("CNY", "AED", TEAL, 3, ls="--", arrow=False)
edge("THB", "AED", TEAL, 3, ls="--", arrow=False)
edge("USD", "CNY", GRAY, 3, rad=-0.28, arrow=False)
edge("USD", "CNY", GOLD, 7, rad=0.12, z=3)
edge("CNY", "THB", GOLD, 7, rad=0.12, z=3)
edge("THB", "USD", GOLD, 7, rad=0.12, z=3)
for k, (x, y) in pos.items():
    ax.add_patch(plt.Circle((x, y), 0.42, color=NAVY if k != "AED"
else "#56606b", zorder=4))
    ax.text(x, y, name[k], ha="center", va="center", color="white",
fontsize=19, weight="bold", zorder=5, linespacing=1.15)
lab = {("USD", "CNY"): (1.55, 0.55), ("CNY", "THB"): (0.0, -0.05),
("THB", "USD"): (-1.55, 0.55)}
for a, b, rate, lr, lf in rows:
    x, y = lab[(a, b)]
    ax.text(x, y, f"log レート{lr:+.4f}\n手数料{lf:+.4f}", ha="center",
va="center", fontsize=18,
            color="#5a4310", bbox=dict(boxstyle="round,pad=0.4",
fc="#fbf3e2", ec=GOLD, lw=1.6), zorder=6)
fig.text(0.5, 0.955, "通貨のグラフ：一周の経路（ドル → 人民元 → タイバーツ → ド
ル）", ha="center", va="top",
         fontsize=26, weight="bold", color=NAVY)
fig.text(0.5, 0.055, "金色の矢印：一周の経路　　灰色の実線：従来の送金網　　青緑
の点線：中央銀行デジタル通貨の決済網",
         ha="center", va="center", fontsize=17, color="#444")
```

```
fig.text(0.98, 0.02, "数値は、物語の中の架空の数値", ha="right",
fontsize=13, color="#777")
fig.savefig("mucam_py01a_currency_graph.jpg", dpi=120,
pil_kwargs={"quality": 92}); plt.close(fig)

# ================ PY-01b：一周の計算の表 ================
fig = plt.figure(figsize=(16, 9), dpi=120)
tx = fig.add_axes([0.06, 0.04, 0.88, 0.86]); tx.axis("off");
tx.set_xlim(0, 1); tx.set_ylim(0, 1); tx.set_autoscale_on(False)
fig.text(0.5, 0.955, "一周の和が 0 からずれる ―― 離散的なホロノミー",
ha="center", va="top", fontsize=28, weight="bold", color=NAVY)
tx.text(0.0, 0.97, "各辺の対数レートと手数料を足し合わせる。ずれがなければ、一周
の和は 0 になる。", fontsize=19, va="top", color="#333")
cols = [0.00, 0.30, 0.52, 0.76]
y = 0.80
for c, h in zip(cols, ["経路", "レート", "log レート", "手数料（log）"]):
    tx.text(c, y, h, fontsize=21, weight="bold", color=NAVY)
for a, b, rate, lr, lf in rows:
    y -= 0.11
    for c, v in zip(cols, [f"{a} → {b}", f"{rate:.4f}", f"{lr:+.4f}",
f"{lf:+.4f}"]):
        tx.text(c, y, v, fontsize=21)
y -= 0.07; tx.plot([0, 1], [y + 0.035, y + 0.035], color="#999",
lw=1.5)
tx.text(0.00, y - 0.04, "一周の和", fontsize=24, weight="bold")
tx.text(0.52, y - 0.04, f"{total:+.4f}", fontsize=26, weight="bold",
color=RED)
tx.text(0.00, y - 0.17, f"1 ドルから出発して一周すると、{1+gain:.4f} ドルで
戻る（{gain*100:+.2f}%）。", fontsize=21, color=RED)
tx.text(0.00, y - 0.30, "経路の網の目（送金網の切断や、新しい決済網）が変わる
と、ずれが現れる場所も変わる。", fontsize=19, color="#333")
fig.text(0.98, 0.02, "数値は、物語の中の架空の数値", ha="right",
fontsize=13, color="#777")
fig.savefig("mucam_py01b_loop_sum.jpg", dpi=120,
pil_kwargs={"quality": 92}); plt.close(fig)
print(f"一周の和 = {total:+.4f}、増減 = {gain*100:+.2f}%")
```

第 3 章　PY-02：平穏な市場と危機の市場の、ユークリッド距離とフィッシャー・ラオ距離

3.1 本編との関係

本編の第 4 章の場面の図を描く。出力する画像：mucam_py02_fisher_vs_euclid.jpg

3.2 実行の結果

コードを実行すると、次の値を表示する。

ユークリッド距離 0.0452、フィッシャー・ラオ距離（式）2.4000、測地線に沿って足した長さ 2.4000

3.3 検算と読み方

閉じた形の式で計算したフィッシャー・ラオ距離と、測地線（x = μ/√2 の座標で、横軸に中心を持つ半円）に沿って計量で長さを足し合わせた値が、どちらも 2.4000 で一致する。これが、本編の数値の検算である。

3.4 コードの全文

```
"""
曲率の裁定（MUCAM）図PY-02：平穏な市場と危機の市場の、分布どうしの距離
平均μ を横軸、標準偏差σ を縦軸とする平面（上半平面）に、
平穏な市場N(0.1%, 1%) と危機の市場N(−2%, 5%) を置き、
まっすぐな線（ユークリッド距離）と、フィッシャー情報計量の測地線（フィッシャー・ラオ距離）を描き分ける。
正規分布の族のフィッシャー情報計量は ds² = (dμ² + 2dσ²)/σ²。x = μ/√2 と置くと、双曲平面（上半平面）の計量の2 倍になる。
"""
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib import font_manager

for f in font_manager.findSystemFonts():
    if "NotoSansCJK" in f and ("Regular" in f or "Bold" in f):
        font_manager.fontManager.addfont(f)
plt.rcParams["font.family"] = "Noto Sans CJK JP"
NAVY, TEAL, GOLD, RED, GRAY = "#1e2a3a", "#1f8a8a", "#c8962e",
"#a0471e", "#7a8590"

m1, s1 = 0.001, 0.01      # 平穏な市場
m2, s2 = -0.02, 0.05      # 危機の市場
d_euc = np.hypot(m1 - m2, s1 - s2)
d_fr = np.sqrt(2) * np.arccosh(1 + ((m1 - m2) ** 2 / 2 + (s1 - s2) **
2) / (2 * s1 * s2))

# 測地線：x = μ/√2 の座標では、x 軸に中心を持つ半円
x1, x2 = m1 / np.sqrt(2), m2 / np.sqrt(2)
c = (x1**2 + s1**2 - x2**2 - s2**2) / (2 * (x1 - x2))
r = np.hypot(x1 - c, s1)
t1, t2 = np.arctan2(s1, x1 - c), np.arctan2(s2, x2 - c)
t = np.linspace(t1, t2, 400)
geo_mu, geo_s = np.sqrt(2) * (c + r * np.cos(t)), r * np.sin(t)
# 検算：測地線に沿って、計量で長さを足し合わせる
dmu, ds = np.diff(geo_mu), np.diff(geo_s); sm = (geo_s[1:] + geo_s[:-
1]) / 2
length = np.sum(np.sqrt(dmu**2 + 2 * ds**2) / sm)
```

```
fig = plt.figure(figsize=(16, 9), dpi=120)
ax = fig.add_axes([0.07, 0.12, 0.55, 0.74])
P = 100  # パーセントで表示
full_t = np.linspace(0.02, np.pi - 0.02, 400)
ax.plot(np.sqrt(2) * (c + r * np.cos(full_t)) * P, r * np.sin(full_t)
* P, color=GOLD, lw=1.5, ls=":", alpha=0.7)
ax.plot([m1 * P, m2 * P], [s1 * P, s2 * P], color=GRAY, lw=3.5,
ls="--", label="まっすぐな線（ユークリッド距離）")
ax.plot(geo_mu * P, geo_s * P, color=GOLD, lw=5, label="フィッシャー情報
計量の測地線")
ax.scatter([m1 * P], [s1 * P], s=260, color=TEAL, zorder=5);
ax.scatter([m2 * P], [s2 * P], s=260, color=RED, zorder=5)
ax.annotate("平穏な市場\nN(0.1%, 1%)", (m1 * P, s1 * P), xytext=(0.6,
1.2), fontsize=19, color=TEAL, weight="bold")
ax.annotate("危機の市場\nN(−2%, 5%)", (m2 * P, s2 * P), xytext=(-3.9,
5.4), fontsize=19, color=RED, weight="bold")
ax.set_xlim(-4.2, 2.2); ax.set_ylim(0, 6.6)
ax.set_xlabel("平均 μ（%）", fontsize=20); ax.set_ylabel("標準偏差 σ（%）",
fontsize=20)
ax.tick_params(labelsize=16); ax.grid(alpha=0.25)
ax.legend(fontsize=16, loc="upper right", frameon=True)
ax.text(-4.05, 0.25, "σ = 0（横軸）は、双曲空間の\"果て\"", fontsize=14,
color="#666")

fig.text(0.5, 0.955, "平穏な市場と危機の市場は、どれくらい遠いか",
ha="center", va="top", fontsize=28, weight="bold", color=NAVY)
tx = 0.66
fig.text(tx, 0.80, "ユークリッド距離", fontsize=21, weight="bold",
color=GRAY)
fig.text(tx, 0.74, f"{d_euc:.3f}", fontsize=34, weight="bold",
color=GRAY)
fig.text(tx, 0.68, "平均と標準偏差の差から計算", fontsize=16,
color="#444")
fig.text(tx, 0.58, "フィッシャー・ラオ距離", fontsize=21, weight="bold",
color=GOLD)
fig.text(tx, 0.52, f"{d_fr:.1f}", fontsize=34, weight="bold",
color=GOLD)
fig.text(tx, 0.455, "データからの\"見分けやすさ\"で測る", fontsize=16,
color="#444")
fig.text(tx, 0.395, "測地線は、標準偏差の大きい側へ\nふくらんで進む。σ が大き
いところでは、\n分布どうしを見分けにくく、\n距離が\"短く\"なるから。",
fontsize=16, color="#333", linespacing=1.6, va="top")
fig.text(tx, 0.08, "２つの距離は、単位も意味も違うので、\n数字の大きさをそのま
ま比べない。", fontsize=15, color=RED, linespacing=1.5)
fig.savefig("mucam_py02_fisher_vs_euclid.jpg", dpi=120,
pil_kwargs={"quality": 92})
print(f"ユークリッド距離 {d_euc:.4f}、フィッシャー・ラオ距離（式）{d_fr:.4f}、
測地線に沿って足した長さ {length:.4f}")
```

第 4 章　PY-03：何の距離で見るかで、空間の形が変わる

4.1 本編との関係

本編の第 4 章の場面の図を描く。出力する画像：mucam_py03_wasserstein_flat.jpg

4.2 実行の結果

コードを実行すると、次の値を表示する。

```
ok
```

4.3 検算と読み方

フィッシャー情報計量の円の上のすべての点が、中心からフィッシャー・ラオ距離でちょうど 0.6 であることを、コードの中の assert で確かめている（確かめに失敗すると、実行が止まる）。

4.4 コードの全文

```
"""
曲率の裁定（MUCAM）図PY-03：何の距離で見るかで、空間の形が変わる
左：フィッシャー情報計量で見た (μ, σ) 平面に、同じ半径（フィッシャー・ラオ距離）の円を描く。σ が小さいほど小さく見える（双曲空間）。
右：ワッサースタイン距離（W2）で見た同じ平面に、同じ半径の円を描く。どこでも同じ大きさ（平ら）。
1 次元の正規分布どうしの W2 距離は sqrt((μ1−μ2)² + (σ1−σ2)²) で、(μ, σ) 平面のユークリッド距離と一致する。
フィッシャー情報計量 ds² = (dμ² + 2dσ²)/σ² のもとで、中心 (μ0, σ0)、半径 ρ の円は、
x = μ/√2 の座標で、中心 (x0, σ0 cosh R)、半径 σ0 sinh R のふつうの円（R = ρ/√2）になる。
"""
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib import font_manager

for f in font_manager.findSystemFonts():
    if "NotoSansCJK" in f and ("Regular" in f or "Bold" in f):
        font_manager.fontManager.addfont(f)
plt.rcParams["font.family"] = "Noto Sans CJK JP"
NAVY, TEAL, GOLD, RED = "#1e2a3a", "#1f8a8a", "#c8962e", "#a0471e"

centers = [(-2.0, 1.0), (0.0, 2.0), (2.0, 4.0)]   # (μ, σ) [%]
th = np.linspace(0, 2 * np.pi, 300)

def fisher_circle(mu0, s0, rho):
    R = rho / np.sqrt(2)
    cx, cy, rr = mu0 / np.sqrt(2), s0 * np.cosh(R), s0 * np.sinh(R)
```

```
    return np.sqrt(2) * (cx + rr * np.cos(th)), cy + rr * np.sin(th)

def fr_dist(m1, s1, m2, s2):
    return np.sqrt(2) * np.arccosh(1 + ((m1 - m2) ** 2 / 2 + (s1 - s2)
** 2) / (2 * s1 * s2))

fig = plt.figure(figsize=(16, 9), dpi=120)
fig.text(0.5, 0.965, "何の距離で見るかで、空間の形が変わる", ha="center",
va="top", fontsize=30, weight="bold", color=NAVY)
for k, (title, sub, col) in enumerate([("フィッシャー情報計量で見る", "同じ
半径（0.6）の円が、σ が小さいほど小さく見える = 双曲空間", GOLD),
                                       ("ワッサースタイン距離で見る", "同じ
半径（0.6%）の円が、どこでも同じ大きさ = 平ら", TEAL)]):
    ax = fig.add_axes([0.06 + k * 0.49, 0.13, 0.42, 0.62]);
ax.set_aspect("equal")
    for (m0, s0) in centers:
        if k == 0:
            x, y = fisher_circle(m0, s0, 0.6)
            assert np.allclose(fr_dist(m0, s0, x, y), 0.6, atol=1e-6)
# 検算：円の上の点は、中心からちょうど0.6
        else:
            x, y = m0 + 0.6 * np.cos(th), s0 + 0.6 * np.sin(th)
        ax.fill(x, y, color=col, alpha=0.25); ax.plot(x, y, color=col,
lw=3)
        ax.scatter([m0], [s0], s=70, color=NAVY, zorder=5)
    ax.set_xlim(-4, 4.5); ax.set_ylim(0, 6.5)
    ax.set_xlabel("平均 μ（%）", fontsize=19); ax.set_ylabel("標準偏差
σ（%）", fontsize=19)
    ax.tick_params(labelsize=15); ax.grid(alpha=0.25)
    fig.text(0.27 + k * 0.49, 0.85, title, ha="center", fontsize=24,
weight="bold", color=col)
    fig.text(0.27 + k * 0.49, 0.80, sub, ha="center", fontsize=15,
color="#333")
fig.text(0.5, 0.035, "同じ正規分布の集まりでも、"
         "見分けやすさで測ると曲がって（双曲空間）、確率の重みを運ぶ手間で測ると平
らに見える。", ha="center", fontsize=16, color="#333")
fig.savefig("mucam_py03_wasserstein_flat.jpg", dpi=120,
pil_kwargs={"quality": 92})
print("ok")
```

第 5 章　PY-04：損失の裾の形が変わる ―― VaR と期待ショートフォール

5.1 本編との関係

本編の第 7 章の場面の図を描く。出力する画像：mucam_py04_tail_and_es.jpg

5.2 実行の結果

コードを実行すると、次の値を表示する。

```
平穏な時期: 閾値 1.16、xi 0.114、beta 0.494、VaR 1.90、ES 2.55、標本の ES 2.56 ／ 送金網が切断された後: 閾値 1.95、xi 0.300、beta 1.025、VaR 3.71、ES 5.93、標本の ES 5.95
```

5.3 検算と読み方

一般化パレート分布の当てはめから計算した期待ショートフォール（2.55%、5.93%）と、標本から直接計算した値（2.56%、5.95%）が、ほぼ一致する。架空の損失は、t 分布から乱数で作っている（乱数の種は固定）。

5.4 コードの全文

```
"""
曲率の裁定（MUCAM）図PY-04：損失の分布の裾 ――VaR の境目と、境目を超えた損失の平均（期待ショートフォール）
平穏な時期と、送金網が切断された後の時期の、日次の損失（%）の分布を並べ、
右の裾（上位10%）に一般化パレート分布を当てはめる（閾値を超えた損失の分析）。
VaR（97.5%）と期待ショートフォール（97.5%）は、当てはめた一般化パレート分布から計算する。数値は架空のもの。
"""
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib import font_manager
from scipy import stats

for f in font_manager.findSystemFonts():
    if "NotoSansCJK" in f and ("Regular" in f or "Bold" in f):
        font_manager.fontManager.addfont(f)
plt.rcParams["font.family"] = "Noto Sans CJK JP"
NAVY, TEAL, GOLD, RED, GRAY = "#1e2a3a", "#1f8a8a", "#c8962e",
"#a0471e", "#9aa3ad"

rng = np.random.default_rng(11)
periods = [("平穏な時期", stats.t(df=6, loc=0.0, scale=0.8).rvs(4000,
random_state=rng), TEAL),
           ("送金網が切断された後", stats.t(df=3, loc=0.2,
scale=1.1).rvs(4000, random_state=rng), RED)]
alpha, q_u = 0.975, 0.90

def evt(losses):
    u = np.quantile(losses, q_u)                       # 閾値（上位10%の境目）
    exc = losses[losses > u] - u
    xi, _, beta = stats.genpareto.fit(exc, floc=0)     # 一般化パレート分布（形xi、尺度beta）
```

```
    p_u = np.mean(losses > u)
    var = u + beta / xi * (((1 - alpha) / p_u) ** (-xi) - 1)
    es = (var + beta - xi * u) / (1 - xi)              # xi < 1 のときの期待ショートフォール
    return u, xi, beta, p_u, var, es

fig = plt.figure(figsize=(16, 9), dpi=120)
fig.text(0.5, 0.985, "損失の裾の形が変わる ―― VaR と期待ショートフォール（97.5%）", ha="center", va="top", fontsize=27, weight="bold", color=NAVY)
for i, (name, L, col) in enumerate(periods):
    u, xi, beta, p_u, var, es = evt(L)
    ax = fig.add_axes([0.06 + i * 0.48, 0.30, 0.42, 0.52])
    bins = np.linspace(-5, 10, 90)
    ax.hist(L, bins=bins, density=True, color=col, alpha=0.30, edgecolor="white")
    xs = np.linspace(u, 10, 300)
    ax.plot(xs, p_u * stats.genpareto.pdf(xs - u, xi, loc=0, scale=beta), color=GOLD, lw=4, label="裾に当てはめた一般化パレート分布")
    ax.axvspan(var, 10, color=RED, alpha=0.10)
    ax.axvline(var, color=NAVY, lw=3, label="VaR（97.5%）"); ax.axvline(es, color=RED, lw=3, ls="--", label="期待ショートフォール")
    ax.set_xlim(-5, 10); ax.set_xlabel("日次の損失（%）", fontsize=17); ax.tick_params(labelsize=13)
    if i == 0: ax.set_ylabel("頻度（密度）", fontsize=17)
    ax.legend(fontsize=13, loc="upper right")
    # 裾の拡大図
    ins = ax.inset_axes([0.55, 0.12, 0.42, 0.45])
    ib = np.linspace(u, 10, 40); tail = L[(L > u) & (L <= 10)]
    ins.hist(tail, bins=ib, weights=np.full(len(tail), 1.0 / (len(L) * (ib[1] - ib[0]))), color=col, alpha=0.30, edgecolor="white")   # 全体の標本数で割った密度（本体の図と同じ尺度）
    ins.plot(xs, p_u * stats.genpareto.pdf(xs - u, xi, loc=0, scale=beta), color=GOLD, lw=3)
    ins.axvspan(var, 10, color=RED, alpha=0.12); ins.axvline(var, color=NAVY, lw=2.5); ins.axvline(es, color=RED, lw=2.5, ls="--")
    ins.set_xlim(u, 10); ins.set_ylim(0, 0.08); ins.tick_params(labelsize=10)
    ins.set_title("裾の拡大", fontsize=13, color="#333")
    cx = 0.27 + i * 0.48
    fig.text(cx, 0.865, name, ha="center", fontsize=23, weight="bold", color=col)
    fig.text(cx, 0.19, f"VaR {var:.2f}%", ha="center", fontsize=24, weight="bold", color=NAVY)
    fig.text(cx, 0.125, f"期待ショートフォール {es:.2f}%", ha="center", fontsize=24, weight="bold", color=RED)
    fig.text(cx, 0.07, f"裾の形のパラメーター ξ = {xi:.2f}（大きいほど裾が厚い）", ha="center", fontsize=15, color="#444")
    print(f"{name}: 閾値 {u:.2f}、xi {xi:.3f}、beta {beta:.3f}、VaR {var:.2f}、ES {es:.2f}、標本の ES
```

```
{L[L>=np.quantile(L,alpha)].mean():.2f}")
fig.text(0.98, 0.01, "数値は架空のもの。損失の上位 10%に一般化パレート分布を当てはめて計算。赤く塗った部分の平均が期待ショートフォール。", ha="right",
fontsize=11.5, color="#777")
fig.savefig("mucam_py04_tail_and_es.jpg", dpi=120,
pil_kwargs={"quality": 92})
```

第 6 章　PY-05：k 形式が測るもの、同じ矢印を 2 本並べると面積は 0

6.1 本編との関係

本編の第 3 章の場面の図を描く。出力する画像：
mucam_py05a_k_forms.jpg、mucam_py05b_collapse.jpg

6.2 実行の結果

コードを実行すると、次の値を表示する。

```
ok
```

6.3 検算と読み方

平行四辺形の面積は、2 本の矢印の成分から計算した行列式の絶対値で、同じ向きの 2 本では 0.00 になる。

6.4 コードの全文

```
"""
曲率の裁定（MUCAM）図PY-05a・PY-05b：k 形式が測るもの
PY-05a：1 形式＝矢印 1 本に数を 1 つ返す（増え方の目盛り）、2 形式＝向きのついた面積、3 形式＝向きのついた体積
PY-05b：同じ矢印を 2 本並べると、平行四辺形はつぶれて面積は 0
"""
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib import font_manager
from matplotlib.patches import Polygon, FancyArrowPatch, Arc
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

for f in font_manager.findSystemFonts():
    if "NotoSansCJK" in f and ("Regular" in f or "Bold" in f):
        font_manager.fontManager.addfont(f)
plt.rcParams["font.family"] = "Noto Sans CJK JP"
NAVY, TEAL, GOLD, RED, GRAY = "#1e2a3a", "#1f8a8a", "#c8962e",
"#a0471e", "#9aa3ad"
```

```
def arrow(ax, p, q, color, lw=4, z=5):
    ax.add_patch(FancyArrowPatch(p, q, arrowstyle="-|>",
mutation_scale=30, color=color, lw=lw, zorder=z))

# ================ PY-05a ================
fig = plt.figure(figsize=(16, 9), dpi=120)
fig.text(0.5, 0.975, "k形式が測るもの", ha="center", va="top",
fontsize=30, weight="bold", color=NAVY)

# --- 左：1形式 ---
ax = fig.add_axes([0.02, 0.10, 0.31, 0.72]); ax.set_aspect("equal");
ax.axis("off"); ax.set_xlim(-0.3, 4.3); ax.set_ylim(-0.6, 4.3)
for k in range(9):                          # 1形式を、等間隔の目盛りの線（等高
線）で表す
    x = 0.45 * k
    ax.plot([x, x + 1.2], [0, 4.0], color=GRAY, lw=1.6, zorder=1)
arrow(ax, (0.2, 1.0), (3.3, 2.0), TEAL)
ax.text(1.8, 2.15, "v", fontsize=26, color=TEAL, weight="bold",
style="italic")
ax.text(2.0, -0.45, "矢印が、目盛りの線を何本横切るか", ha="center",
fontsize=17, color="#333")
fig.text(0.175, 0.845, "1形式", ha="center", fontsize=26,
weight="bold", color=NAVY)
fig.text(0.175, 0.80, "矢印1本＝数1つ（増え方）", ha="center",
fontsize=18, color="#333")

# --- 中：2形式 ---
ax = fig.add_axes([0.35, 0.10, 0.30, 0.72]); ax.set_aspect("equal");
ax.axis("off"); ax.set_xlim(-0.4, 4.4); ax.set_ylim(-0.6, 4.3)
o, u, w = np.array([0.3, 0.6]), np.array([3.0, 0.6]), np.array([0.9,
2.6])
ax.add_patch(Polygon([o, o + u, o + u + w, o + w], closed=True,
fc="#f3dfb4", ec=GOLD, lw=2.5, zorder=1))
arrow(ax, o, o + u, TEAL); arrow(ax, o, o + w, RED)
ax.text(*(o + u * 0.55 + np.array([0, -0.45])), "v", fontsize=26,
color=TEAL, weight="bold", style="italic")
ax.text(*(o + w * 0.5 + np.array([-0.5, 0])), "w", fontsize=26,
color=RED, weight="bold", style="italic")
c = o + (u + w) / 2
ax.add_patch(Arc(c, 1.3, 1.3, theta1=20, theta2=320, color=NAVY,
lw=2.5))
ax.add_patch(FancyArrowPatch(c + 0.65 *
np.array([np.cos(np.radians(300)), np.sin(np.radians(300))]),
                             c + 0.65 *
np.array([np.cos(np.radians(320)), np.sin(np.radians(320))]),
                             arrowstyle="-|>", mutation_scale=22,
color=NAVY, lw=2.5))
ax.text(2.0, -0.45, "回る向きのついた面積", ha="center", fontsize=17,
color="#333")
fig.text(0.50, 0.845, "2形式", ha="center", fontsize=26, weight="bold",
```

```
color=NAVY)
fig.text(0.50, 0.80, "矢印２本－向きのついた面積", ha="center",
fontsize=18, color="#333")

# --- 右：3 形式 ---
ax = fig.add_axes([0.67, 0.10, 0.31, 0.72], projection="3d");
ax.set_axis_off()
o = np.zeros(3); a, b, cc = np.array([1.0, 0, 0]), np.array([0.35,
1.0, 0]), np.array([0.2, 0.25, 1.0])
V = [o, a, a + b, b, cc, a + cc, a + b + cc, b + cc]
faces = [[V[0], V[1], V[2], V[3]], [V[4], V[5], V[6], V[7]], [V[0],
V[1], V[5], V[4]],
         [V[2], V[3], V[7], V[6]], [V[1], V[2], V[6], V[5]], [V[0],
V[3], V[7], V[4]]]
ax.add_collection3d(Poly3DCollection(faces, facecolor="#cfe3e3",
edgecolor="#5a6b78", linewidth=1.4, alpha=0.30))
for vec, col, lab in [(a, TEAL, "u"), (b, RED, "v"), (cc, GOLD, "w")]:
    ax.quiver(0, 0, 0, *vec, color=col, linewidth=5,
arrow_length_ratio=0.15, zorder=10)
    ax.text(*(vec * 1.18 + np.array([0.0, 0.0, 0.03])), lab,
fontsize=26, color=col, weight="bold", style="italic")
ax.set_xlim(0, 1.5); ax.set_ylim(0, 1.4); ax.set_zlim(0, 1.2);
ax.view_init(elev=22, azim=-60)
fig.text(0.825, 0.845, "3 形式", ha="center", fontsize=26,
weight="bold", color=NAVY)
fig.text(0.825, 0.80, "矢印３本－向きのついた体積", ha="center",
fontsize=18, color="#333")
fig.text(0.825, 0.13, "向きのついた体積", ha="center", fontsize=17,
color="#333")
fig.text(0.98, 0.02, "n 次元の空間では、n 形式まである", ha="right",
fontsize=15, color="#555")
fig.savefig("mucam_py05a_k_forms.jpg", dpi=120, pil_kwargs={"quality":
92}); plt.close(fig)

# ================= PY-05b =================
fig = plt.figure(figsize=(16, 9), dpi=120)
fig.text(0.5, 0.94, "同じ矢印を２本並べると、平行四辺形はつぶれて、面積は 0",
ha="center", va="top", fontsize=28, weight="bold", color=NAVY)
for i, (ang, title) in enumerate([(55, "２本の向きが違う"), (25, "２本の向
きが近い"), (0, "同じ向き（v と v）")]):
    ax = fig.add_axes([0.03 + i * 0.32, 0.12, 0.30, 0.66]);
ax.set_aspect("equal"); ax.axis("off"); ax.set_xlim(-0.3, 4.3);
ax.set_ylim(-1.0, 2.6)
    o = np.array([0.1, 0.3]); u = np.array([2.5, 0.0]); w = 1.5 *
np.array([np.cos(np.radians(ang)), np.sin(np.radians(ang))])
    if ang == 0: w = np.array([2.5, 0.0])
    o = o + np.array([0, 0.6])
    ax.add_patch(Polygon([o, o + u, o + u + w, o + w], closed=True,
fc="#f3dfb4", ec=GOLD, lw=2.5))
    if ang == 0:
```

```
        arrow(ax, o + np.array([0, 0.12]), o + u + np.array([0,
0.12]), TEAL); arrow(ax, o - np.array([0, 0.12]), o + w - np.array([0,
0.12]), RED, lw=3)
        ax.text(2.75, 0.72, "v", fontsize=24, color=TEAL,
weight="bold", style="italic"); ax.text(2.75, 0.22, "v", fontsize=24,
color=RED, weight="bold", style="italic")
    else:
        arrow(ax, o, o + u, TEAL); arrow(ax, o, o + w, RED, lw=3)
    area = abs(u[0] * w[1] - u[1] * w[0])
    ax.text(1.9, -0.75, f"面積 = {area:.2f}", ha="center", fontsize=22,
color=RED if area < 1e-9 else NAVY, weight="bold")
    fig.text(0.18 + i * 0.32, 0.80, title, ha="center", fontsize=21,
color="#333")
fig.text(0.5, 0.05, "だから 2 形式では、v と v を並べると 0。順番を入れ替えると、
符号が反対になる（交代的）。", ha="center", fontsize=19, color="#333")
fig.savefig("mucam_py05b_collapse.jpg", dpi=120,
pil_kwargs={"quality": 92}); plt.close(fig)
print("ok")
```

第 7 章　PY-06：相関が 1 に近づくと、散らばりの「体積」がつぶれる

7.1 本編との関係

本編の第 4 章の場面の図を描く。出力する画像：mucam_py06_volume_collapse.jpg

7.2 実行の結果

コードを実行すると、次の値を表示する。

```
ok
```

7.3 検算と読み方

標準偏差がどちらも 1%のとき、共分散行列の行列式は 1 − 相関^2（1、0.36、0.0199）。95%の範囲の楕円の面積は π × 5.991 × √（行列式）（18.8、11.3、2.7）。5.991 は自由度 2 のカイ二乗分布の 95%点。

7.4 コードの全文

```
"""
曲率の裁定（MUCAM）図PY-06：相関が1 に近づくと、散らばりの「体積」（共分散行列の行
列式）がつぶれる
２つの銘柄の日次リターン（標準偏差はどちらも 1%）の散布図を、相関0、0.8、0.99 で描
き、
95%の範囲を表す楕円を重ねる。楕円の面積は、共分散行列の行列式の平方根に比例する。数
値は架空のもの。
"""
import numpy as np
```

```python
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib import font_manager
from matplotlib.patches import Ellipse

for f in font_manager.findSystemFonts():
    if "NotoSansCJK" in f and ("Regular" in f or "Bold" in f):
        font_manager.fontManager.addfont(f)
plt.rcParams["font.family"] = "Noto Sans CJK JP"
NAVY, TEAL, GOLD, RED = "#1e2a3a", "#1f8a8a", "#c8962e", "#a0471e"

rng = np.random.default_rng(7)
s = 1.0                          # 標準偏差1%（単位：%）
k = np.sqrt(5.991)               # 2次元正規分布の95%の範囲（カイ二乗、自由度2）
fig = plt.figure(figsize=(16, 9), dpi=120)
fig.text(0.5, 0.965, "相関が1 に近づくと、散らばりの\"体積\"がつぶれる",
ha="center", va="top", fontsize=29, weight="bold", color=NAVY)
for i, rho in enumerate([0.0, 0.8, 0.99]):
    C = np.array([[s * s, rho * s * s], [rho * s * s, s * s]])
    X = rng.multivariate_normal([0, 0], C, size=600)
    det = np.linalg.det(C)
    w, V = np.linalg.eigh(C)
    ang = np.degrees(np.arctan2(V[1, 1], V[0, 1]))
    area = np.pi * k * k * np.sqrt(det)          # 楕円の面積 = π k² √det
    ax = fig.add_axes([0.04 + i * 0.325, 0.27, 0.29, 0.58]);
ax.set_aspect("equal")
    ax.scatter(X[:, 0], X[:, 1], s=9, color=TEAL, alpha=0.45)
    ax.add_patch(Ellipse((0, 0), 2 * k * np.sqrt(w[1]), 2 * k *
np.sqrt(max(w[0], 1e-12)), angle=ang,
                         fc=GOLD, alpha=0.25, ec=GOLD, lw=3))
    ax.set_xlim(-4.2, 4.2); ax.set_ylim(-4.2, 4.2)
    ax.set_xlabel("銘柄Aのリターン（%）", fontsize=16)
    if i == 0: ax.set_ylabel("銘柄Bのリターン（%）", fontsize=16)
    ax.tick_params(labelsize=13); ax.grid(alpha=0.25)
    cx = 0.185 + i * 0.325
    fig.text(cx, 0.165, f"相関{rho:.2f}", ha="center", fontsize=26,
weight="bold", color=NAVY)
    fig.text(cx, 0.10, f"行列式{det:.4f}（%⁴）", ha="center",
fontsize=22, weight="bold", color=RED if rho > 0.9 else NAVY)
    fig.text(cx, 0.05, f"楕円の面積{area:.1f}（%²）", ha="center",
fontsize=17, color="#444")
fig.text(0.98, 0.005, "数値は架空のもの。楕円は95%の範囲。面積 = π × 5.991 ×
√(行列式)", ha="right", fontsize=12, color="#777")
fig.savefig("mucam_py06_volume_collapse.jpg", dpi=120,
pil_kwargs={"quality": 92})
print("ok")
```

第 8 章　PY-07：大きな輪のずれを、三角形ごとに分ける（離散版のストークスの定理）

8.1 本編との関係

本編の第 8 章の場面の図を描く。出力する画像：
mucam_py07a_big_loop_graph.jpg、mucam_py07b_loop_breakdown.jpg

8.2 実行の結果

コードを実行すると、次の値を表示する。

```
三角形１ 3、三角形２ 7、穴 -74、大きな輪 -64
```

8.3 検算と読み方

大きな輪の一周のずれ（−64 bp）が、三角形１（+3）、三角形２（+7）、穴のまわり（−74）の和にちょうど分かれることを、コードの中の assert で確かめている。内側の辺の値は、隣り合う２つの部分で逆向きに数えられるので、打ち消し合う。

8.4 コードの全文

```
"""
曲率の裁定（MUCAM）図PY-07a・PY-07b：大きな輪のずれを、三角形ごとに分ける（離散版のストークスの定理）
両替の対数レートのずれ（理論値からの差、単位：ベーシスポイント、1bp = 0.01%）を、辺の上の値（1 形式）として置く。
大きな輪 ドル→人民元→ディルハム→バーツ→ドル のずれは、
三角形がそろっている部分（三角形ごとの一周のずれ）と、三角形で埋められない部分（穴のまわりのずれ）の和に分かれる。
人民元とバーツの経路は切断されたまま（点線）で、ユーロとディルハムの経路もないため、ユーロ・人民元・ディルハム・バーツに囲まれた部分は三角形で埋められない。
数値は架空のもの。
"""
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib import font_manager
from matplotlib.patches import Polygon, FancyArrowPatch

for f in font_manager.findSystemFonts():
    if "NotoSansCJK" in f and ("Regular" in f or "Bold" in f):
        font_manager.fontManager.addfont(f)
plt.rcParams["font.family"] = "Noto Sans CJK JP"
NAVY, TEAL, GOLD, RED, GRAY = "#1e2a3a", "#1f8a8a", "#c8962e",
"#a0471e", "#9aa3ad"

# ---- 辺の上の値（向きのある辺。逆向きは符号が反対）----
sig = {("USD", "CNY"): 12, ("CNY", "EUR"): -5, ("EUR", "USD"): -4,
```

```
        ("EUR", "THB"): 9, ("THB", "USD"): -6,
        ("CNY", "AED"): 8, ("AED", "THB"): -78}
def s(a, b): return sig[(a, b)] if (a, b) in sig else -sig[(b, a)]
def loop(*v): return sum(s(v[i], v[(i + 1) % len(v)]) for i in
range(len(v)))
T1 = loop("USD", "CNY", "EUR")           # 三角形1
T2 = loop("USD", "EUR", "THB")           # 三角形2
H  = loop("EUR", "CNY", "AED", "THB")    # 穴のまわり
L  = loop("USD", "CNY", "AED", "THB")    # 大きな輪
assert L == T1 + T2 + H                  # 離散版のストークスの定理による分解
の検算

pos = {"USD": (-1.6, 1.4), "CNY": (1.6, 1.4), "AED": (1.6, -1.4),
"THB": (-1.6, -1.4), "EUR": (-0.75, 0.25)}
name = {"USD": "ドル", "CNY": "人民元", "AED": "ディルハム", "THB": "バー
ツ", "EUR": "ユーロ"}

# ================= PY-07a：グラフ=================
fig = plt.figure(figsize=(16, 9), dpi=120)
ax = fig.add_axes([0.03, 0.07, 0.94, 0.80]); ax.set_aspect("equal");
ax.axis("off"); ax.set_xlim(-3.3, 3.3); ax.set_ylim(-1.9, 1.9)
P = lambda k: np.array(pos[k])
ax.add_patch(Polygon([P("USD"), P("CNY"), P("EUR")], fc="#cfe3e3",
ec="none", zorder=0))
ax.add_patch(Polygon([P("USD"), P("EUR"), P("THB")], fc="#f3dfb4",
ec="none", zorder=0))
for a, b in [("USD", "EUR"), ("CNY", "EUR"), ("THB", "EUR")]:
    ax.plot(*zip(pos[a], pos[b]), color=GRAY, lw=3, zorder=1)
ax.plot(*zip(pos["CNY"], pos["THB"]), color=RED, lw=3, ls=(0, (4, 4)),
zorder=1)
for a, b in [("USD", "CNY"), ("CNY", "AED"), ("AED", "THB"), ("THB",
"USD")]:
    ax.add_patch(FancyArrowPatch(pos[a], pos[b], arrowstyle="-|>",
mutation_scale=30, color=GOLD, lw=6, shrinkA=34, shrinkB=34,
zorder=2))
for k, (x, y) in pos.items():
    ax.add_patch(plt.Circle((x, y), 0.30, color=NAVY, zorder=4))
    ax.text(x, y, name[k], ha="center", va="center", color="white",
fontsize=17 if len(name[k]) < 5 else 14, weight="bold", zorder=5)
ax.text(*((P("USD") + P("CNY") + P("EUR")) / 3), f"三角形1\n{T1:+d}
bp", ha="center", va="center", fontsize=20, weight="bold", color=TEAL)
ax.text(*((P("USD") + P("EUR") + P("THB")) / 3 + np.array([0.08,
0.0])), f"三角形2\n{T2:+d} bp", ha="center", va="center", fontsize=20,
weight="bold", color="#8a6a20")
ax.text(0.85, -0.35, "穴", ha="center", va="center", fontsize=44,
weight="bold", color=RED)
ax.text(0.85, -0.85, "三角形で埋められない", ha="center", va="center",
fontsize=15, color=RED)
ax.text(-0.45, -0.95, "人民元—バーツ（切断されたまま）", ha="center",
va="center", fontsize=14, color=RED, rotation=41)
```

```
fig.text(0.5, 0.955, "大きな輪のずれを、三角形ごとに分ける", ha="center",
va="top", fontsize=28, weight="bold", color=NAVY)
fig.text(0.5, 0.04, "金色の矢印：大きな輪（ドル→人民元→ディルハム→バーツ→ドル）　塗った三角形：３辺がそろっている部分　赤の点線：切断された経路",
         ha="center", fontsize=14, color="#444")
fig.text(0.985, 0.01, "数値は架空のもの。bp（ベーシスポイント）＝0.01%",
ha="right", fontsize=11.5, color="#777")
fig.savefig("mucam_py07a_big_loop_graph.jpg", dpi=120,
pil_kwargs={"quality": 92}); plt.close(fig)

# ================ PY-07b：内訳の表 ================
fig = plt.figure(figsize=(16, 9), dpi=120)
tx = fig.add_axes([0.08, 0.05, 0.84, 0.82]); tx.axis("off");
tx.set_xlim(0, 1); tx.set_ylim(0, 1); tx.set_autoscale_on(False)
fig.text(0.5, 0.955, "大きな輪のずれの内訳", ha="center", va="top",
fontsize=29, weight="bold", color=NAVY)
rows = [("三角形１（ドル・人民元・ユーロ）", T1, TEAL, "どの通貨の組が出どころか、分かる"),
        ("三角形２（ドル・ユーロ・バーツ）", T2, "#8a6a20", "どの通貨の組が出どころか、分かる"),
        ("三角形ごとのずれの合計", T1 + T2, NAVY, ""),
        ("穴のまわりに残るずれ", H, RED, "１つの通貨の組のせいにはできない")]
y = 0.84
for lab, v, c, note in rows:
    tx.text(0.0, y, lab, fontsize=22, weight="bold" if "合計" in lab or
"穴" in lab else "normal", color=c, va="center")
    tx.text(0.62, y, f"{v:+d} bp", fontsize=26, weight="bold",
color=c, va="center")
    if note: tx.text(0.0, y - 0.065, note, fontsize=15, color="#555",
va="center")
    y -= 0.17
tx.plot([0, 1], [y + 0.07, y + 0.07], color="#999", lw=1.5)
tx.text(0.0, y - 0.02, "大きな輪の一周のずれ", fontsize=24,
weight="bold", color=NAVY, va="center")
tx.text(0.62, y - 0.02, f"{L:+d} bp", fontsize=30, weight="bold",
color=NAVY, va="center")
tx.text(0.0, y - 0.12, f"＝三角形ごとの合計（{T1+T2:+d}）＋穴のまわりのずれ（{H:+d}）", fontsize=19, color="#333", va="center")
fig.text(0.985, 0.01, "数値は架空のもの。bp（ベーシスポイント）＝0.01%",
ha="right", fontsize=11.5, color="#777")
fig.savefig("mucam_py07b_loop_breakdown.jpg", dpi=120,
pil_kwargs={"quality": 92}); plt.close(fig)
print(f"三角形１ {T1}、三角形２ {T2}、穴 {H}、大きな輪 {L}")
```

第 9 章　実行の環境

執筆者は、Python 3.12.3、numpy 2.5.4、scipy 1.18.1、matplotlib 3.11.2 の環境で、 7 本すべてが正しく実行され、上の値を表示し、画像を出力することを確かめた（2026 年 11 月）。